Harness Engineering

Birgitta Böckeler

Birgitta is a Distinguished Engineer and AI-assisted delivery expert at Thoughtworks. She has over 20 years of experience as a software developer, architect and technical leader.

This article is part of "Exploring Gen AI". A series capturing Thoughtworks technologists' explorations of using gen ai technology for software development.

17 February 2026

It was very interesting to read OpenAI's recent write-up on "Harness engineering" which describes how a team used "no manually typed code at all" as a forcing function to build a harness for maintaining a large application with AI agents. After 5 months, they've built a real product that's now over 1 million lines of code.

The article is titled "Harness engineering: leveraging Codex in an agent-first world", but only mentions "harness" once in the text. Maybe the term was an afterthought inspired by Mitchell Hashimoto's recent blog post. Either way, I like "harness" as a word to describe the tooling and practices we can use to keep AI agents in check.

The OpenAI team's harness components mix deterministic and LLM-based approaches across 3 categories (grouping based on my interpretation):

1. **Context engineering**: Continuously enhanced knowledge base in the codebase, plus agent access to dynamic context like observability data and browser navigation
2. **Architectural constraints**: Monitored not only by the LLM-based agents, but also deterministic custom linters and structural tests
3. **"Garbage collection"**: Agents that run periodically to find inconsistencies in documentation or violations of architectural constraints, fighting entropy and decay

They also highlight how iterative this is: "When the agent struggles, we treat it as a signal: identify what is missing — tools, guardrails, documentation — and feed it back into the repository, always by having Codex itself write the fix."

All of the described measures focus on increasing long-term internal quality and maintainability. What I am missing in the write-up is verification of functionality and behaviour.

Leaving that gap aside, and assuming we can trust OpenAI's representation of the success of this (with respect to the author and the team, OpenAI do have a vested interest in us believing in AI-maintainable code) — here are my thoughts on what *is* in the article.

Harnesses - the future service templates?

Most organizations have just two or three main tech stacks — not every application is its own snowflake. The article made me imagine a future where teams pick from a set of harnesses for common application topologies to get started. This evokes today's service templates, which help teams instantiate new services on a "golden path". Will harnesses — with custom linters, structural tests, basic context and knowledge documentation, and additional context providers — become the new service templates? Will teams use them as a starting point, then shape them over time for their application's specifics?

With service templates, teams contribute back as they gain experience, then other teams often struggle to incorporate updates. Would we see a similar forking and synchronization challenge with harnesses?

The article also made me revisit some of my older hypotheses:

The runtime has to be constrained for more AI autonomy?

A lot of early and current AI coding hype assumes LLMs will give us unlimited flexibility of the target runtime. Generate in any language, any pattern, without constraints — the LLM will figure it out. But for maintainable, AI-generated code at scale that we can trust, something has to give.

The harness described suggests that increasing trust and reliability required constraining the solution space: specific architectural patterns, enforced boundaries, standardized structures. That means giving up some "generate anything" flexibility for prompts, rules, and harnesses full of technical specifics.

A convergence on a limited number of tech stacks and topologies?

As coding becomes less about typing code and more about steering its generation, AI might push us toward fewer tech stacks. Usability of frameworks and SDKs still matters

— we're seeing repeatedly that what's good for humans is good for AI. But developer tastes will matter less at that level of detail. Little inefficiencies and idiosyncracies in interfaces will be less annoying since we don't deal with them directly. We might choose stacks with good harnesses available and prioritize "AI-friendliness".

This might apply not just to tech stacks, but also to codebase structures and topologies. We might default to structures that are easier to maintain with AI because they're easier to harness. The OpenAI team discusses architectural rigidity and enforcement rules. The main focus areas I can see are keeping data structures stable and defining and enforcing module boundaries. Sounds reasonable — but without concrete examples, I'm still struggling to imagine what "we require Codex to parse data shapes at the boundary" looks like in practice in their harness.

But if we can figure out widely how to harness codebase design patterns, will these topologies become the new abstraction layer, not natural language itself like so many AI enthusiasts hope?

Two future worlds: Pre-AI vs post-AI application maintenance?

Say we develop good harnessing techniques to turn AI autonomy up to 9 and increase our confidence in the results. Which techniques could we apply to existing applications, and which would only work for applications built from scratch with a harness in mind?

For older codebases, we'd need to consider whether retrofitting a harness is worth the effort. AI can help us do that faster, but those applications are often so non-standardized and full of entropy that it might not be worthwhile. It makes me think of running a static code analysis tool on a codebase that's never had one, and then drowning in alerts.

What's your harness today?

That this team worked on their harness for 5 months shows this isn't something you can jump into for quick results. But it's worth reflecting on what your harness is today. Do you have a pre-commit hook? What's in it? Do you have ideas for custom linters? What architectural constraints would you like to impose on your codebase? Have you experimented with structural testing frameworks like ArchUnit?

Final thoughts

Unsurprisingly, what they describe sounds like much more work than just generating and maintaining a bunch of Markdown rules files. They built extensive tooling for the

deterministic part of the harness. Their context engineering involved not only curating a knowledge base, but also significant design work — the code design itself is a huge part of the context.

The OpenAI team says: “Our most difficult challenges now center on designing environments, feedback loops, and control systems.” This reminded me of Chad Fowler’s recent post on “Relocating Rigor”. It’s refreshing to hear concrete ideas and experiences about where that rigor might go, rather than just hoping “better models” will magically solve maintainability issues.

And finally, for once, I like a term in this space. Though it’s only 2 weeks old — I can probably hold my metaphorical breath until somebody calls their one-prompt, LLM-based code review agent a harness…

latest article (Feb 17):
Harness Engineering

previous article:
Context Engineering for Coding Agents

© Martin Fowler | Disclosures